Reflection Naming: Reification

Document #:	P2088R0
Date:	2020-01-12
Project:	Programming Language C++ SG7

Reply-to:	Mihail Naydenov <mihailnajdenov@gmail.com>

§ Abstract

This paper suggests alternative naming for the reification operators, part of the Reflection API[1], with the aim to greatly increase consistency and discoverability.

§ Issue

Reification is the reverse of reflection, turning a reflection object into program code. Because one reflection object can represent few different program code entities, multiple keywords are needed to do reification. Here is the complete list, as presented in the Reflection paper:

> `typename(reflection)` A simple-type-specifier corresponding to the type designated by “reflection”. Ill-formed if “reflection” doesn’t designate a type or type alias.
>
> `namespace(reflection)` A namespace-name corresponding to the namespace designated by “reflection”. Ill-formed if “reflection” doesn’t designate a namespace.
>
> `template(reflection)` A template-name corresponding to the template designated by “reflection”. Ill-formed if “reflection” doesn’t designate a template.
>
> `valueof(reflection)` If “reflection” designates a constant expression, this is an expression producing the samevalue (including value category). Otherwise, ill-formed.
>
> `exprid(reflection)` If “reflection” designates a function, parameter or variable, data member, or an enumerator, this is equivalent to an id-expression referring to the designated entity (without lookup, access control, or overload resolution: the entity is already identified). Otherwise, this is ill-formed.
>
> `[: reflection :]` OR `unqualid(reflection)` If “reflection” designates an alias, a named declared entity, this is an identifier referring to that alias or entity. Otherwise, ill-formed.

> `[< reflection >]` OR `templarg(reflection)` Valid only as a template argument. Same as “typename(reflection)” if that is well-formed. Otherwise, same as “template(reflection)” if that is well-formed. Otherwise, same as “valueof(reflection)” if that is well-formed. Otherwise, same as “exprid(reflection)”.

Looking at all these as a whole, it is not hard for one to notice, there is little to no consistency b/w them, meaning:

— It is hard to understand, the operators are part of the same framework.

— It is hard to understand, the operators are part of “reflection”.

— It is hard to find all or alternative operators after seeing only few of them.

What is more, there is no “simple” or “smart” option, one which would work in the absence of ambiguity. We could easily imagine scenarios where there is only one possible (or “most correct”) result from a reification, and these scenarios are not really covered.

> For example, we could agree, that given `type(r)`, assuming `r` is some reflection object and `type` returns the type reflection object, there exist expected and unambiguous result from the reification of that object, and that is the concrete type. Right now we must be extra specific, that we want type reification and not something else - `typename(type(r))`. This can be seen as redundant.

Besides overall inconsistency and lack of simpler options, some of the operators have issues on their own. I will look at each one separately.

typename(reflection)

This one is clever, but the fact it reuses a keyword for a different action comes with problems.

First and foremost, *parenthesis changing the meaning of a keyword is contra the established practice!* Take a look at `sizeof` - it can be called with and without parenthesis, depending on the parameter, and it does the same thing.
With and without parenthesis, `typename` will do radically different things. We could argue that in both cases, “a type is introduced”, but that does not change the fact these two are radically different, both conceptually and in term of implementation.

Second. By reusing the keyword, we “pretend”, reification of a type is the same as regular name introduction. This is of questionable value. Arguably, we want reification to be *glaring obvious* in code as it can completely change its meaning - we don’t want to confuse `typename (X::something)` with a `typename X::something`.

Third, the name of the keyword is technically incorrect, as it does not introduce *a name* - the semantics are much closer to that of `decltype`, not `typename`. We could even imagine a potential confusion, people expecting `typename(r)` to return the name of the type (as a string).

Forth, the confusion with existing keywords will get even worse with metaclasses,[2] as `class(something)` will be used to introduce them. Considering `typename` and `class` are often interchangeable, people will inevitably confuse `class(property)` with `typename(property)`.

Fifth? Using reification in template code can be hard to read and understand - `typename T::typename(template X<T>::B)`. (Added a question mark, because I am not sure if the first `typename` is needed.) It is worth further noting, `template` can do reification as well.

namespace(reflection)

From all reifiers, reusing keywords, this one is least problematic, with least chance of causing confusion. Then again, we might have to write `using namespace namespace(something);`, which is not exactly self-explanatory.

template(reflection)

Similar to `typename`, overloading this keyword will not do us favors. Arguably here it is even worse, simply because `template` *is already heavily overloaded:*
Class templates, function templates, specializations, disambiguation within templates, `template for`, with angle brackets, without brackets, with empty brackets - there is no end! If we add `template` with pareths as well, it will be simply too much.

valueof(reflection)

This reifier is fine, on its own, it does what it says. Of course, if one is not aware of reflections it could mean anything.

exprid(reflection) and **unqualid(reflection)**

These two use the C++ Standard lingo to say “name” - an “id” (“identifier”). This is not user-friendly (more like “expert-friendly”) and contra to the existing usage of the term “id” - `typeid`, `thread::id`, `locale::id`, etc.

Differentiating b/w these two also requires high-tier knowledge about both Reflection and the C++ language.

templarg(reflection)

This is the only reifier, used in just one specific place and its usage is embedded into its name. This might serve us at the moment, but what if we find other places where its result is useful, some place other then template argument?

[: reflection :] and [< reflection >]

These two create very, very funky looking code.

```
int [:reflexpr(name):] = 42;
C::[:C::r():] y;
X<[<... t_args>]> x;
```

We are definitely stepping into “looking as another language” territory. What is worse, because the lack of a keyword, reification becomes *intertwined* with regular code, making it hard to reason about at a glance.

Lack of names also makes these hard to search for on the Internet or in a documentation. One must know and remember the “academic” name, used in the Standard, in order to find information about them.

§ Proposed solution

To have *both* consistency *and* discoverability, we introduce a keyword that states the action it performs - `reify`.

`reify` will come in two flavors.

— single keyword, used in all places where there is no ambiguity what is to be reified.

— `reify_*` series of keywords for all cases, not handled by the single form.

The single form will be used when it is “obvious” what the intend is, acting as the “simple” or “smart” solution, described above:

```
int i; constexpr auto r = reflof(i);

reify(type(r)) j;       //< int, because of type(r)
float reify('_', r);    //< `_i`, because the concatenation overload
      works on id-s directly
...
for (auto m : members(reflof(C)))
  reify(m) = {};        //< member of C, because of members()
```

The idea is, `reify` should work in all places where if it *doesn’t*, it would feel pedantic or verbose.

For all cases where `reify` alone is ambiguous *or* the user whats a specific result, we will have specialized keywords, like in the current proposal:

`reify_type(r)` or `reify_t(r)`, in pace of `typename(r)`.

`reify_namespace(r)` or `reify_ns(r)`, in pace of `namespace(r)`.

`reify_template(r)` or `reify_tmp(r)`, in pace of `template(r)`.

`reify_value(r)` or `reify_v(r)`, in pace of `valueof(r)`.

`reify_name(r)` or `reify_nm(r)` or the geeky `reify_id(r)`, in pace of `unqualid(r)`.

`reify_any(r)` as it literally does that, testing any possibility, or the eventually `reify_targ(r)`, in pace of `templarg(r)`.

> The author seeks guidance, which naming alternative to choose. Having multiple versions at the same time is not proposed.

For `exprid(r)` we could just use the regular `reify`, anticipating this will be the most commonly used operation - to reify "the thing" - the member, the function, the variable, etc. In other words, `reify` should try to be as smart and as useful as possible, effectively always returning something - either unambiguously *or* defaulting to "the entity". If this is not feasible, we could think of a concrete reifier like `reify_entity` or `reify_ent`.

Realizing the above, the framework becomes consistent not just into itself, but also to a framework, already in the language - the casting ensemble of keywords: `static_cast`, `dynamic_cast`, `const_cast`, `reinterpret_cast`.

This symmetry b/w casting and reification is not an unwelcome one. Reification *can* be viewed as a form of casting *from* reflection object *to* program code, and in both cases one object can be cast to multiple different things. Overall, both subsytems will have the same benefits (and arguably similar downsides, like verboseness) - in particular the *great* discoverability for both humans and tools, making it impossible to confuse what the code does:

Before

```
...
typename (C::r) x;
C::[:C::r:] y;
```

After

```
...
reify_type(C::r) x;     //< or
      reify_t
C::reify_name(C::r) y;  //< or
      reify_id
```

Additionally, because we don't overload keywords, we could use reifiers without parenthesizes:

Before

```
...
template<typename T1, typename T2>
using
mp_assign = typename
      (rf_assign(reflexpr(T1),
      reflexpr(T2)));
```

Notice how close this construct is to "dependent lookup" use of 'typename'.

After

```
...
template<typename T1, typename T2>
using
mp_assign = reify_type
      rf_assign(reflof(T1),
      reflof(T2));
```

Less verbose and glaringly clear, we do reification.

§ Conclusion

Having all reifiers behind similar syntax solves all the problems listed at the beginning:

— Easy to understand, the operators are part of the same framework. If we invent a new reifier, it will also have “a place” - we will not have to worry about keyword or symbol availability.

— Easy to understand, the operators are part of “reflection”.

— Easy to find all or alternative operators after seeing only few of them.

— Have a simple solution for the simple cases.

By adopting such naming schema we shift from heavily intertwined reification code to strongly separated, brightly highlighted one.

The need for a simple, “smart” option is not to be underestimated as well. *All* reflection proposals *initially* envision *just one* reifier (in Reflection, called `unreflexpr`). Although, this proves unfeasible as a complete solution, we should not go to the other extreme and end up with an API that feels overly pedantic or redundant.

---

1. Reflection: http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2019/p1240r1.pdf↩

2. Metaclasses: http://www.open-std.org/JTC1/SC22/WG21/docs/papers/2019/p0707r4.pdf↩